

CampusCert

A Secure, Transparent SaaS Examination Platform

Semester Project Guide — Online Assessment & Examination Platform

5-Member Software Engineering Team | 6-8 Week Delivery Window

Inspired by the ClassMarker product concept — not a clone.

Table of Contents

1. Executive Summary.....	4
2. Project Vision.....	5
3. Problem Statement.....	6
4. Existing Solutions / ClassMarker Inspiration.....	7
5. Proposed Solution.....	8
6. Target Users.....	9
7. Value Proposition.....	10
8. Business Model.....	11
9. User Roles.....	12
10. Functional Requirements — MVP Feature List.....	13
11. Non-Functional Requirements.....	15
12. MVP Scope & Priority System.....	16
13. AI-Assisted Question Generation.....	17
14. Secure Examination & Integrity Monitoring.....	18
15. Examiner Control Room.....	19
16. Result Transparency & Audit Trail.....	20
17. Complete User Workflows.....	21
18. System Architecture.....	22
19. Database Design.....	23
20. ER Diagram.....	24
21. API Architecture.....	25
22. Frontend Architecture.....	26
23. Repository Architecture.....	27
24. Git / GitHub Workflow.....	28
25. Commit Convention.....	29
26. GitHub Issues & Project Board.....	30
27. Git Security Precautions.....	31
28. Team Work Distribution.....	32
29. 6–8 Week Development Roadmap.....	33
30. Testing Strategy.....	34
31. Deployment Strategy.....	35
32. Final Demo Scenario (10–15 minutes).....	36
33. Future Scope.....	37
34. Explicitly Out of Scope for the Core MVP.....	38
35. Risks & Mitigation.....	39
36. What Makes This Project Impressive.....	40

37. Final MVP Definition & Checklist.....	41
38. Team Collaboration Rules.....	42
39. Project Name Suggestions.....	43
40. Conclusion.....	44

1. Executive Summary

CampusCert is a SaaS-based online examination platform built for a five-person, six-to-eight-week university project. It is inspired by the business concept behind ClassMarker — question banks, configurable exams, automated grading, and organizational accounts — but it is scoped down to what five students can realistically design, build, test, and demo alongside a normal course load.

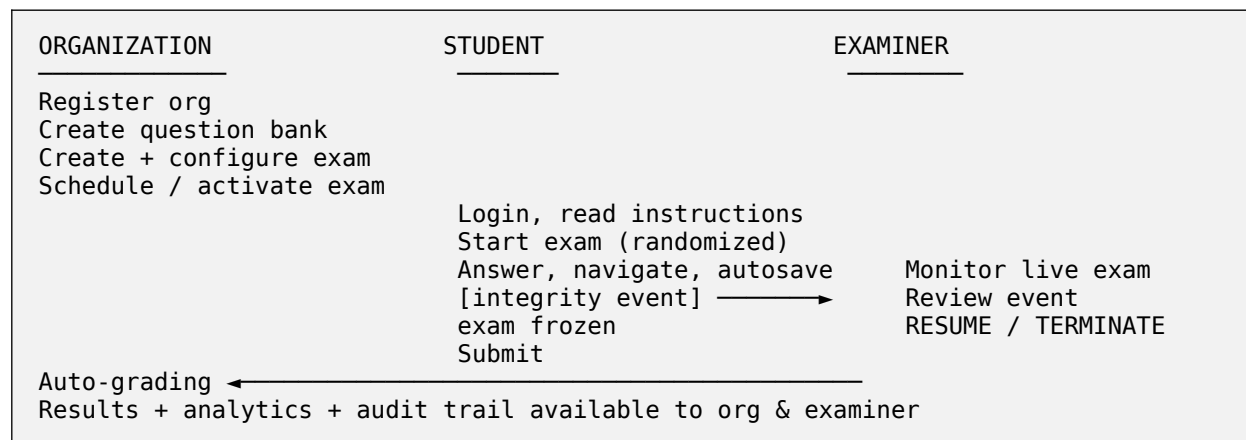
The product focuses on one concrete use case first: secure, randomized entry/admission testing for universities and training organizations, with two things most quiz tools skip — a temporary-freeze integrity workflow (instead of silent auto-fail) and a full per-answer audit trail that can defend a result if a candidate disputes it.

The guiding rule throughout this document is MVP-first: every feature is labeled P0/P1/P2/Future, and anything that would blow the 6-week budget is explicitly pushed to Future Scope rather than quietly assumed.

2. Project Vision

Enable any institution — university, training center, or certification body — to run secure, auditable, auto-graded online exams without building examination infrastructure themselves.

End-to-end workflow



3. Problem Statement

Institution-side problems

- Question banks are hard to build and maintain at scale.
- Preparing multiple unique papers for large candidate pools is manual and slow.
- Manual grading does not scale and introduces error.
- Managing candidates, schedules, and results across spreadsheets is error-prone.
- Online exam integrity is hard to enforce, and disputes have no audit record to fall back on.
- Building this infrastructure in-house is expensive for a single institution.

Candidate-side problems

- Need a reliable, clearly-timed exam interface with unambiguous instructions.
- Results often feel opaque; there is no way to verify how a score was produced.
- If a dispute arises, there is typically no record of what happened during the attempt.

Note: this document does not claim any specific university's admission-test system is faulty or fraudulent. Public complaints about entry-test results are referenced only as reported student concerns, not proven technical failures. The motivating problem is the general lack of transparent, auditable, scalable assessment infrastructure — not an accusation against any institution.

4. Existing Solutions / ClassMarker Inspiration

ClassMarker is used here as a reference point for the product category, not as a specification to copy. The concept we are borrowing is: an organization account creates a reusable question bank, assembles configurable tests from it, assigns them to candidates, and gets automatic grading plus results — as a hosted service rather than custom-built software.

Category	What it typically offers	Gap CampusCert targets
Simple quiz apps (Kahoot-style)	Fun, low-stakes quizzes, live play	No integrity controls, no audit trail, not exam-grade
LMS (Moodle, Canvas)	Course content + basic quizzing bolted on	Assessment is secondary; weak security/audit for high-stakes exams
Generic online assessment SaaS	Question banks, auto-grading, orgs	Often thin on dispute-defensible auditability
Dedicated secure exam platforms	Proctoring, lockdown browsers	Enterprise pricing/complexity, overkill for a single institution's admission test
CampusCert (this project)	Question banks, randomized exams, freeze-and-review integrity flow, full audit trail, org accounts	Positioned specifically for university admission-style testing at student-project scale

5. Proposed Solution

CampusCert is a modular-monolith web application with three interfaces sharing one backend and one database:

- University/Admin Portal — question banks, exam configuration, scheduling, candidate management, analytics.
- Student Exam Portal — instructions, timed exam interface, autosave, submission, results.
- Examiner Control Room — live monitoring, integrity event review, resume/terminate, audit trail inspection.

6. Target Users

Segment	Examples
Primary	Universities, colleges, educational institutes, training organizations
Secondary	Corporate training departments, certification providers, tutoring/coaching centers, recruitment/assessment firms
Candidates	University applicants, enrolled students, trainees, job applicants, certification candidates

7. Value Proposition

CampusCert is positioned as secure, transparent SaaS examination infrastructure for organizations — not a quiz app and not a full LMS.

- Vs. quiz apps: exam-grade timing, randomization, and integrity controls instead of casual play.
- Vs. LMS assessment modules: assessment is the core product, not an afterthought, so security and auditability go deeper.
- Vs. enterprise secure-exam platforms: right-sized for a single institution's admission testing, without enterprise pricing or lockdown-browser complexity.
- Why organizations would rather subscribe than build: examination infrastructure (grading logic, randomization, integrity workflow, audit storage) is a specialized, ongoing engineering cost most institutions have no reason to own.

8. Business Model

Plan tiers (business narrative — not all tiers are built in the MVP)

Plan	Illustrative features	Built in semester MVP?
Free / Basic	Limited exams, question bank, MCQs, auto-grading, basic results, limited candidates	Yes — this is effectively the MVP feature set
Professional	Bulk import, advanced analytics, randomization, multiple admins, certificates, branding, integrity monitoring, control room, audit trail, AI question generation, paid exams	Partially — bulk import, randomization, control room, audit trail, and a limited AI-assist are built; certificates/branding/multi-admin billing are not
Enterprise	Custom domain, API integrations, advanced proctoring, institutional SSO, dedicated support	No — described as future vision only

Monetization narrative (business plan, not implemented as real billing)

- Organization subscriptions (per-plan monthly/annual fee).
- Usage/credit-based pricing for exam volume.
- Paid examinations/certifications with an optional platform transaction fee.
- Future: enterprise contracts, API/integration pricing.

For the semester build, any "paid exam" flow uses a sandbox/simulated payment step (a mock checkout that flips a status flag). Integrating a real payment gateway is explicitly out of scope — it adds compliance and testing overhead the timeline cannot absorb.

9. User Roles

Four roles, kept intentionally simple:

Role	Core capabilities
Platform Admin	Oversees the SaaS platform itself and its organizations (thin layer in the MVP — mainly org approval/visibility)
Organization Admin	Manages question banks, exams, candidates, schedules, results, analytics for their organization
Examiner	Monitors live exams, reviews integrity events, resumes/terminates attempts, reviews results and audit trail
Student / Candidate	Registers, accesses assigned exams, takes exams, submits, views permitted results

If four roles prove to be too much surface area for the team, Platform Admin and Organization Admin can be merged for the MVP demo (single "Admin" login), with the Platform-vs-Org distinction reintroduced only if time allows.

10. Functional Requirements — MVP Feature List

Authentication

- Registration/login
- Password hashing
- JWT-based sessions
- Role-based access control

Organization

- Organization account
- Organization dashboard

Question Bank

- Create/edit/delete MCQ questions with options and correct answer
- Subject, topic, difficulty metadata
- Search and filter

Bulk Import

CSV/Excel import with columns: Question | Option A | Option B | Option C | Option D | Correct Answer | Subject | Topic | Difficulty. This matters because a real institution may have thousands of questions and manual entry is not viable.

Exam Creation & Configuration

- Name, duration, number of questions
- Subject/topic distribution, difficulty mix
- Start/end time, attempt limit

Randomization

- Random question selection per candidate
- Random option order
- Different effective question sets per candidate

Student Exam Interface

- Timer
- Question navigation (next/previous, jump-to)
- Unanswered indicator, mark-for-review
- Autosave
- Submit

Automatic Grading

- Total marks, correct/incorrect/unanswered counts, percentage/score

Results

- Student sees permitted result info; admin/examiner see full results

Audit Trail (key differentiator)

For every attempt, store candidate ID, exam ID, attempt/session ID, question ID, selected answer, correct answer, timestamp, and exam events — enough to reconstruct exactly what happened during the attempt.

11. Non-Functional Requirements

Category	MVP target
Performance	Smooth demo with a few hundred concurrent simulated candidates; no formal load testing at production scale
Reliability	Autosave every answer change so a dropped connection loses at most a few seconds of work
Security	Hashed passwords, JWT expiry, RBAC on every endpoint, no secrets in source control
Auditability	Every graded answer and every integrity event is logged and queryable
Usability	Clear instructions, visible timer, obvious submit/confirm flow
Maintainability	Modular monolith with clearly separated modules per team member
Honesty in claims	The product never claims to prevent all cheating — see Section 16

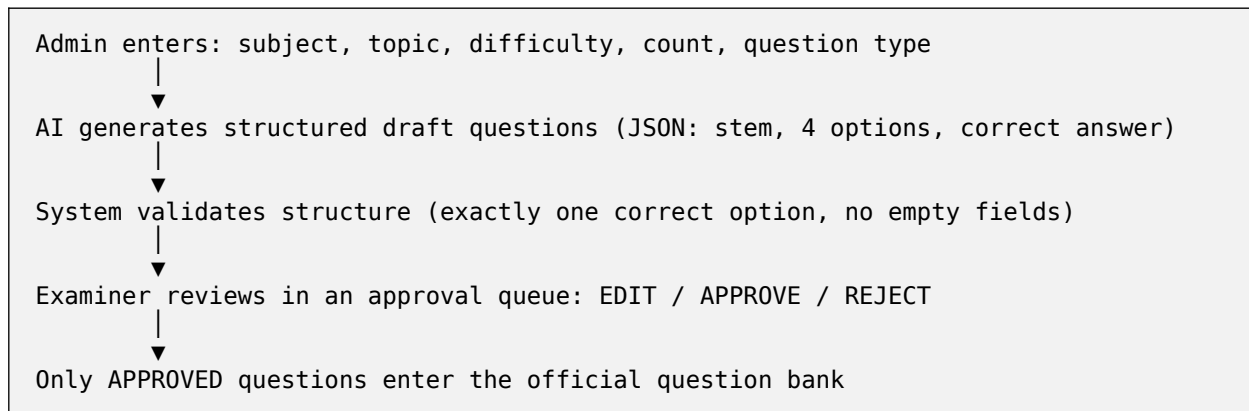
12. MVP Scope & Priority System

Every feature is classified P0 (must have — the demo fails without it), P1 (important, adds real value), P2 (premium/future, only if P0/P1 are stable), or Future (explicitly out of the 6–8 week build).

Feature	Priority
Authentication (register/login, JWT, RBAC)	P0
Question bank CRUD	P0
Bulk CSV/Excel import	P0
Exam creation & configuration	P0
Randomized question/option selection	P0
Student exam interface + timer + autosave	P0
Automatic grading	P0
Results (student + admin view)	P0
Audit trail (answers + events)	P0
Basic integrity detection (tab switch, fullscreen exit, copy/paste)	P1
Freeze / examiner review / resume-terminate	P1
Examiner control room dashboard	P1
Basic analytics (score distribution, subject breakdown)	P1
AI-assisted question generation (human-approved)	P1/P2
Basic webcam face-presence check	P2 — only if P0/P1 are done early
Certificates, branding, multi-admin billing	P2
Real payment gateway integration	P2 — sandbox only in MVP
Advanced AI proctoring / behavior analysis	Future
Native mobile apps	Future
Secure browser / OS-level lockdown	Future
Microservices / Kubernetes	Future — explicitly excluded

13. AI-Assisted Question Generation

AI assists question drafting; it never publishes an official question unadvised. Workflow:



Known risks to flag explicitly in the project write-up: hallucinated or incorrect answer keys, ambiguous wording, two plausibly-correct options, off-syllabus content, API cost/rate limits. This is why human approval is mandatory rather than optional — the platform never lets AI output reach a live exam unreviewed.

MVP implementation: a single "Generate" endpoint calling an LLM API with a structured-JSON prompt, feeding a review queue UI. No fine-tuning, no custom model — an off-the-shelf API call is sufficient and appropriate for the timeline.

14. Secure Examination & Integrity Monitoring

Honest technical framing

A normal browser cannot guarantee full control over a candidate's computer. The platform does not claim to "prevent cheating." The accurate claim is: the platform detects configured examination-integrity events and can temporarily freeze the exam for examiner review. True OS-level lockdown would require a dedicated secure browser or desktop client, which is out of scope.

MVP scope

- Fullscreen-exit detection
- Tab/window switch (visibility/focus loss) detection
- Copy/paste restriction
- Violation event logging
- Temporary exam freeze on a configured violation
- Examiner review queue with Resume / Terminate actions
- Basic webcam face-presence check, only if time allows after P0/P1 are stable

Explicitly future / enterprise

- Advanced AI behavior/movement analysis, object or second-device detection
- Advanced face recognition and liveness detection
- Full screen-recording infrastructure
- Secure browser / OS-level lockdown

Student-facing freeze message

"Your examination has been temporarily paused because an examination-integrity event was detected. Please wait for examiner review."

Freezing instead of auto-failing matters: accidental events happen (an accidental keyboard shortcut, a stray Alt-Tab), and an irreversible auto-fail would be unfair. Human review keeps the system defensible.

15. Examiner Control Room

A polling-based (not real-time-infrastructure-heavy) monitoring dashboard. A simple periodic refresh (e.g., every few seconds) is sufficient for a semester demo and avoids building websocket/event-streaming infrastructure.

LIVE EXAM CONTROL ROOM				
Candidates:	248	Active:	221	
Completed:	24	Flagged:	4	Frozen: 1
Candidate #10245				
Status: REVIEW REQUIRED				
Event: Tab switch detected				
Time: 10:42:17				
Evidence: available				
[RESUME EXAM] [TERMINATE ATTEMPT]				

16. Result Transparency & Audit Trail

This is the platform's strongest differentiator versus a generic quiz tool: a defensible, replayable record of every attempt.

Candidate: C10245 Exam: Engineering Entry Test Attempt: A8891

Q1 -> B -> Correct -> 10:02:14
Q2 -> C -> Incorrect -> 10:02:31
Q3 -> A -> Correct -> 10:02:49
Q4 -> Unanswered -> 10:03:20

10:15:41 Tab switch detected
10:15:41 Exam frozen
10:16:10 Examiner reviewed
10:16:18 Exam resumed

If a candidate disputes a result, the organization can reconstruct exactly what was answered, when, and what integrity events (if any) occurred — instead of taking the system's final score on faith.

17. Complete User Workflows

Organization

Register -> create organization -> build/import question bank -> create exam -> configure exam -> schedule/activate -> candidates take exam -> automatic grading -> results -> analytics/audit.

Student

Register/apply -> access assigned exam -> read instructions -> start exam -> answer questions -> submit -> automatic evaluation -> result.

Examiner/Admin

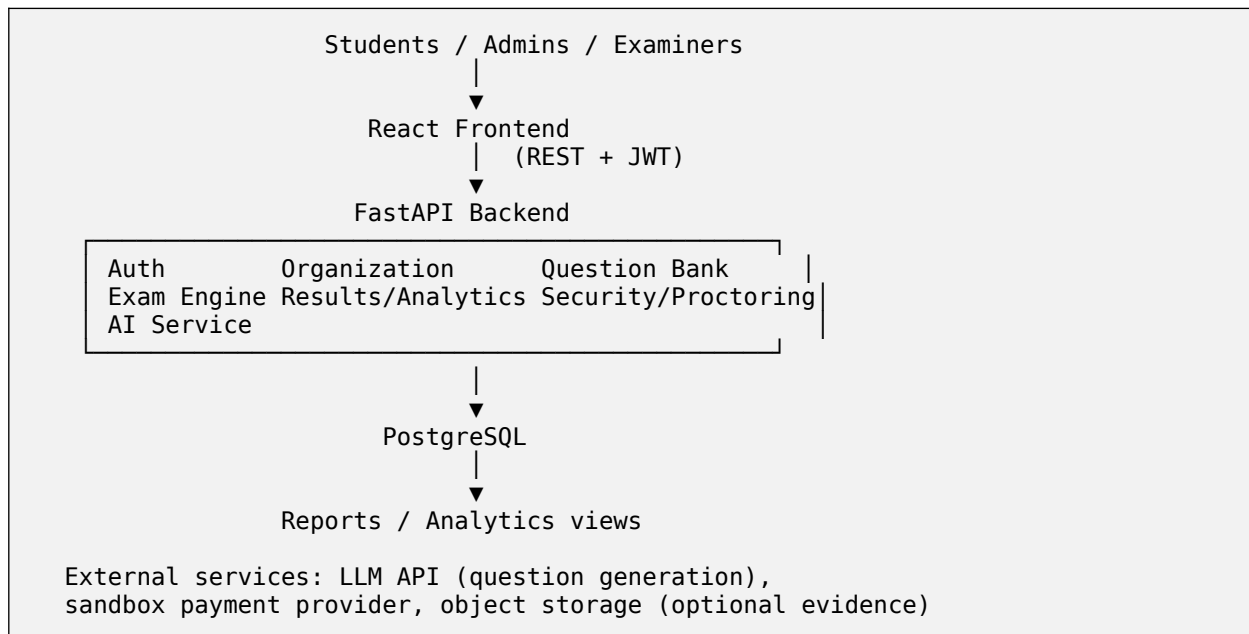
Create/manage questions -> configure exams -> monitor candidates -> receive integrity alerts -> review violations -> resume/terminate attempts -> inspect audit trail -> analyze results.

18. System Architecture

Stack: React (frontend) + FastAPI (backend) + PostgreSQL (database) + JWT (auth) + an LLM API for AI-assisted question generation + simple object/file storage if evidence capture is implemented. Deployed as a single modular monolith, not microservices.

Why a modular monolith

- One codebase, one deployment — far less operational overhead for 5 students in 6–8 weeks.
- Module boundaries (auth, org, question bank, exam engine, security, AI, results) still keep ownership clean per team member.
- Microservices/Kubernetes would spend scarce weeks on infrastructure instead of features.



19. Database Design

MVP entities

Entity	Purpose
User	Login identity, hashed password, role
Organization	Institution/account owning question banks and exams
CandidateProfile	Candidate-specific fields linked to a User
Subject / Topic	Question categorization
Question / QuestionOption	Question stem, options, correct answer, difficulty
Exam / ExamConfiguration	Exam metadata: duration, distribution, timing, attempt limit
ExamQuestion	Resolved set of questions assigned to a specific exam instance/attempt
ExamAttempt	One candidate's session on one exam
CandidateAnswer	Selected answer per question per attempt
ExamEvent	Integrity events (tab switch, freeze, resume, terminate) with timestamps
Result	Computed score/summary per attempt

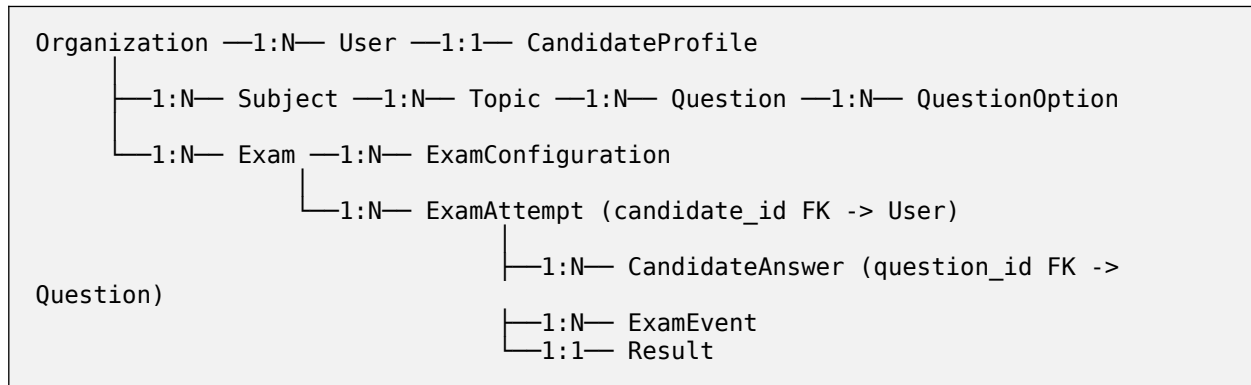
Future-only entities

- Subscription/Plan
- Payment/Transaction (real gateway)
- Certificate
- AIQuestionGenerationRequest (kept minimal or merged into Question in MVP)

Key relationships & constraints

- Organization 1—N Question, 1—N Exam, 1—N User (org members)
- Exam 1—N ExamAttempt; ExamAttempt 1—N CandidateAnswer and 1—N ExamEvent
- Question 1—N QuestionOption; exactly one option flagged correct (app-level constraint)
- Indexes: Question(subject_id, topic_id, difficulty) for fast randomized selection; ExamAttempt(exam_id, candidate_id) unique per attempt-limit rule

20. ER Diagram



21. API Architecture

REST over HTTPS, JSON payloads, JWT bearer auth, versioned under /api/v1. Endpoints grouped by module to match ownership:

- /auth — register, login, refresh
- /organizations — org CRUD, members
- /questions — CRUD, bulk-import, search/filter
- /exams — CRUD, configuration, scheduling
- /attempts — start, autosave-answer, submit, status
- /events — integrity event report, freeze/resume/terminate
- /results — per-attempt result, org-level analytics
- /ai/questions — generate, list pending, approve/reject

22. Frontend Architecture

Single React app with route-based code splitting across three portals (Admin, Student, Examiner), sharing an auth context, an API client, and a component library. State kept simple: React Query (or equivalent) for server state, local component state for UI, no heavyweight global-state framework needed at this scale.

23. Repository Architecture

```
campuscert/
├── backend/           FastAPI app, modules per feature
├── frontend/         React app (admin/student/examiner routes)
├── database/         migrations, seed scripts
├── docs/             architecture, ERD, API docs
├── .github/
│   ├── workflows/    CI (lint, test)
│   ├── ISSUE_TEMPLATE/
│   └── pull_request_template.md
├── .gitignore
├── README.md
├── CONTRIBUTING.md
└── .env.example
```

File/Folder	Purpose
backend/, frontend/, database/	Module code, kept separate so ownership per member is unambiguous
.github/workflows/	CI: lint + automated tests run on every PR
.github/ISSUE_TEMPLATE/, pull_request_template.md	Consistent issue/PR reporting across 5 people
.gitignore	Excludes node_modules, build output, .env, __pycache__, etc.
.env.example	Documents required environment variables with placeholder (non-secret) values
README.md	Setup, run, and demo instructions
CONTRIBUTING.md	The team agreement in Section 27

24. Git / GitHub Workflow

```
main
  ▲ (release: PR from develop, tagged)
develop
  ▲ (PR + 1 reviewer + CI green)
feature/*  fix/*  docs/*
```

- Nobody pushes directly to main.
- Ideally nobody pushes directly to develop — features land via PR.
- Every PR needs at least one reviewer and a passing CI run before merge.
- Rebase/merge latest develop into your feature branch regularly to avoid painful conflicts.
- Never commit secrets or .env; never force-push a shared branch; keep PRs small.

Core commands

```
# initial clone
git clone <REPO_URL>
cd campuscert

# start a feature
git checkout develop
git pull origin develop
git checkout -b feature/exam-timer

# commit
git add .
git commit -m "feat: implement exam timer"

# push
git push -u origin feature/exam-timer

# keep branch current (safest for a student team)
git checkout develop && git pull origin develop
git checkout feature/exam-timer
git merge develop          # resolve conflicts locally, then commit

# open PR: feature/exam-timer -> develop
# release: PR develop -> main, then tag (e.g., v0.3.0)
```

Merge develop into main at clear milestones — end of Week 3 (core flow working end-to-end) and end of Week 6 (MVP freeze) — rather than continuously, so main always reflects a demoable state.

25. Commit Convention

```
feat:    new feature
fix:     bug fix
docs:    documentation only
test:    adding/adjusting tests
refactor: code change with no behavior change
chore:    tooling, config, housekeeping
```

Examples:

```
feat: implement JWT authentication
feat: add bulk question import
fix: correct result calculation
test: add exam submission tests
docs: update database architecture
```

A consistent, meaningful history makes it possible to find when and why a bug was introduced, and gives the final report a clean, professional-looking commit log to show the teacher.

26. GitHub Issues & Project Board

BACKLOG -> TODO -> IN PROGRESS -> IN REVIEW -> DONE

#21 Implement JWT authentication
#22 Create university dashboard
#23 Implement question import
#24 Build exam timer
#25 Add tab-switch detection
#26 Implement audit trail

Every task the team commits to gets an issue, assigned to one owner and one milestone (week). This is what keeps five people, each with their own class schedule, synchronized without constant meetings — anyone can check the board and know exactly what is in progress and what is blocked.

27. Git Security Precautions

Never commit

- API keys
- JWT signing secrets
- Database passwords
- .env files
- Cloud credentials
- Payment/sandbox credentials
- Private certificates

Practices

- .env.example lists variable names only, never real values
- Enable branch protection on main and develop
- Require at least one PR review before merge
- Enable 2FA on the GitHub org/repo
- Turn on secret scanning if available on the plan
- Keep dependencies updated; don't ignore Dependabot alerts
- Never force-push shared branches
- Tag releases at milestones for easy rollback

28. Team Work Distribution

Member	Owns	P0 items owned
1 — Core Backend & Auth	FastAPI setup, DB integration, JWT, RBAC, org/user APIs, shared utilities	Auth, Organization API
2 — Admin Portal	University dashboard, exam creation/configuration, scheduling, candidate management	Exam Creation
3 — Question Bank & AI	Question CRUD, subjects/topics/difficulty, bulk import, random selection, AI-assist + approval	Question Bank, Bulk Import
4 — Student Portal	Student dashboard, instructions, exam interface, timer, autosave, submission, results view	Student Exam, Timer, Auto Grading UI
5 — Security, Proctoring & Analytics	Integrity detection, event logging, freeze/resume/terminate, control room, analytics, audit trail	Audit Trail (shared with 1), Control Room

Member 5 has the most conceptually novel work (integrity detection + control room), so the plan front-loads P0 audit-trail logging as a shared responsibility with Member 1 (who already owns the database layer), and treats webcam/face-presence detection as P2 — only attempted once the P0 items across all five members are done. This keeps Member 5's guaranteed workload comparable to everyone else's, with the more exploratory pieces as stretch goals rather than requirements.

Everyone contributes to testing, documentation, integration, bug fixing, the final presentation, and deployment — these are not any one person's job.

29. 6-8 Week Development Roadmap

Week	Focus	Key deliverables
1	Foundation	Requirements, architecture, ERD, wireframes, GitHub setup, backend/frontend/DB skeletons, authentication
2	Admin + Question Bank	Org dashboard, question CRUD, subjects/topics, bulk import, basic exam creation
3	Exam Engine	Exam configuration, randomized selection, student exam interface, timer, autosave, submission — MERGE TO MAIN
4	Results + Security	Auto grading, results, audit trail, fullscreen/tab detection, violation logging, freeze/resume
5	AI + Analytics + Control Room	AI question generation MVP + examiner approval, control room, analytics, webcam check if feasible
6	Integration & MVP Freeze	End-to-end integration, testing, bug fixing, security pass, UI polish, deployment, demo data — MERGE TO MAIN, TAG RELEASE
7-8 (buffer, optional)	Polish only	Improved AI, better analytics, documentation, presentation rehearsal, deployment hardening — nothing here is required for the core demo

The project must be demo-ready by the end of Week 6. Weeks 7-8 are buffer/polish and must never be load-bearing for the core flow.

30. Testing Strategy

- Unit tests for grading logic, randomization, and RBAC checks (highest risk of silent bugs).
- Integration tests for the full attempt lifecycle: start -> answer -> autosave -> submit -> grade.
- Manual test script for the integrity flow: trigger event -> freeze -> examiner resume/terminate -> verify audit log entries.
- CI runs lint + unit tests on every PR; integration tests run at minimum before each weekly merge to develop.
- A short manual QA pass by a teammate who did not write that module, before every merge to main.

31. Deployment Strategy

A single, simple cloud deployment is enough: one backend service, one frontend build (static hosting or served by the same host), one managed PostgreSQL instance. No blue/green deploys, no multi-region setup, no autoscaling configuration — those are solved problems the timeline should not spend hours re-solving. Environment variables (DB URL, JWT secret, LLM API key) are supplied via the hosting platform's secret store, never committed.

32. Final Demo Scenario (10–15 minutes)

1. University Admin logs in.
2. Loads/creates a question bank.
3. Imports questions via CSV/Excel.
4. Creates "Engineering Admission Test" — 100 questions, 120 minutes.
5. System randomizes the question set.
6. Student logs in and starts the exam.
7. Student answers questions; autosave visibly runs.
8. A tab-switch (or similar) integrity event is simulated.
9. Exam freezes with the student-facing message.
10. Examiner sees the event in the Control Room.
11. Examiner reviews and clicks Resume.
12. Student finishes and submits.
13. System auto-grades instantly.
14. Admin views score, subject-level performance, and the full audit trail.
15. AI question generation is demonstrated: topic + difficulty in, draft questions out, examiner approves one into the bank.

33. Future Scope

Clearly out of the 6–8 week build, but worth naming as the path from student project to a plausible commercial product:

- Advanced AI proctoring: behavior analysis, object/device detection, liveness detection
- Secure browser / OS-level lockdown
- Native mobile apps
- Real payment gateway integration and subscription billing engine
- Enterprise APIs, SSO, institutional integrations
- Custom domains and white-labeling
- Advanced certificate design and issuance
- Large-scale/distributed architecture for very high candidate volumes
- Adaptive testing, multilingual exams, advanced fraud detection

34. Explicitly Out of Scope for the Core MVP

- Native Android/iOS apps
- Microservices or Kubernetes
- Advanced AI proctoring or custom model training
- Sophisticated computer vision / behavior analysis
- OS-level lockdown
- Multiple real payment gateways
- Dedicated SMS/email infrastructure beyond what's strictly needed
- Complex subscription billing engine
- Enterprise-scale infrastructure or 100,000+ candidate load testing
- Full certificate designer
- Full LMS features
- Video lectures, live classes, chat, social features, or recommendation systems

These may be mentioned as future vision but must never become a development requirement for the 6-week core build.

35. Risks & Mitigation

Risk	Mitigation
Scope creep toward a ClassMarker clone	Hold every new idea against the P0/P1/P2/Future table before building it
One member (Security/Proctoring) overloaded	Split audit-trail logging with Member 1; treat webcam detection as P2 stretch goal
Integration left too late	Merge to develop weekly; full merge/tag at end of Week 3 and Week 6
AI generation produces wrong/ambiguous questions	Mandatory human approval before any AI question is official
Team overclaims security ("prevents all cheating")	Use the exact honest framing in Section 14 in all docs and the demo script
Secrets leaked via Git	Branch protection, .env.example only, pre-commit awareness, no direct pushes to main
Timeline slips due to coursework	6-week core scope with weeks 7–8 as true buffer, not hidden requirement

36. What Makes This Project Impressive

- It is not a toy quiz app — it implements a genuine SaaS product shape: organizations, question banks, configurable exams, and role-based portals.
- The freeze-and-review integrity workflow is more honest and more defensible than a silent auto-fail, and shows real product judgment.
- The audit trail turns "trust us" into "here is exactly what happened," which is the single strongest differentiator versus a basic quiz tool.
- AI question generation with mandatory human approval demonstrates responsible use of AI rather than a gimmick.
- The impressiveness comes from the pieces working together end-to-end live in the demo, not from a long feature list.

37. Final MVP Definition & Checklist

If only six weeks are available, this is the exact minimum product that must work, live, end-to-end:

Admin Login -> Create Question Bank -> Bulk Import Questions -> Create Exam
-> Configure Exam -> Student Login -> Start Randomized Exam -> Timer + Autosave
-> Security Event -> Exam Freeze -> Examiner Review -> Resume -> Submit
-> Automatic Grading -> Transparent Result -> Audit Trail -> Analytics

Checklist

- [] Auth + RBAC working for all four roles
- [] Question bank CRUD + bulk CSV/Excel import
- [] Exam creation/configuration with subject/difficulty distribution
- [] Randomized question and option selection per candidate
- [] Student exam UI: timer, navigation, mark-for-review, autosave, submit
- [] Automatic grading producing a stored Result
- [] Full audit trail: every answer and every integrity event, timestamped
- [] At least fullscreen-exit and tab-switch detection, with freeze -> examiner resume/terminate
- [] Examiner control room showing live status and the review queue
- [] Basic analytics view (score distribution / subject breakdown)
- [] AI-assisted question generation with a human approval step
- [] Deployed instance reachable for the live demo
- [] Clean develop -> main history with tagged MVP release

If this complete flow works reliably end-to-end, the project is a successful MVP — regardless of which P2/Future items did not make it in.

38. Team Collaboration Rules

16. Pull latest develop before starting work.
17. One feature = one branch.
18. Never work directly on main.
19. Keep PRs small and focused.
20. Test before opening a PR.
21. Explain what changed and why in the PR description.
22. Review at least one other member's PR per week.
23. Communicate breaking changes before merging them.
24. Resolve conflicts together, not by force-pushing over someone.
25. Don't leave uncommitted work sitting for more than a day or two.
26. Integrate frequently rather than in one big bang at the end.
27. Never commit secrets.
28. Every member should be able to explain their own module to the teacher.
29. Everyone contributes to final integration and testing, not just their own slice.

39. Project Name Suggestions

Name	Why it works
CampusCert	Suggests both campus/education focus and certified/trustworthy results (used as the working name above)
Proctorly	Direct, product-sounding, signals the integrity/monitoring angle
Veritest	"Veritas" (truth) + test — leans into transparency/auditability
ExamLedger	"Ledger" evokes the audit-trail differentiator specifically
TrueMark	Short, brandable, implies accurate/trustworthy grading
Assessa	Clean SaaS-style name, generic enough to grow beyond admission testing
ClearScore Exams	Emphasizes transparency; note: check for naming collisions with existing consumer apps before final use
IntegriTest	On-the-nose but memorable for a teacher-facing pitch

40. Conclusion

CampusCert takes the ClassMarker product concept — organizational question banks, configurable exams, and automated grading — and re-scopes it around what five software engineering students can actually build, test, and confidently demo in six to eight weeks alongside a full course load.

The differentiators that make it feel like a real product rather than a class assignment are deliberately narrow and achievable: randomized exams, an honest freeze-and-review integrity workflow instead of a silent auto-fail, a full per-answer audit trail, and a human-approved AI assist for question drafting. Everything else — advanced proctoring, mobile apps, real payments, enterprise features — is named explicitly as future scope so it never quietly becomes a Week 6 requirement.

Small enough to finish. Professional enough to impress. Complete enough to demonstrate. Scalable enough to explain future potential.